

Operating Systems

Virtual Filesystem

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

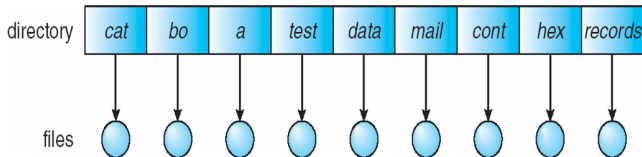
File Concept and Access Methods

- **Contiguous logical** address space
- Can contain any kind of data (numbers, characters, executable code, opaque binary blobs, ...)
- Characterized by attributes
 - **Name** — only information kept in human-readable form
 - **Identifier** — unique identifier within the file system
 - **Type** — needed for systems that support different types
 - **Location** — pointer to file location on device
 - **Size** — current file size
 - **Protection** — controls who can do reading, writing, executing
 - **Ownership** — needed for multiuser systems
 - **Timestamps** — creation, last modification, last access
 - ...

File Types — Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information

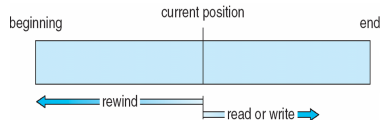
- Information about files are kept in the **directory structure**
- Directory structure: a collection of entities containing information about files
- Both the directory structure and the files reside on disk



- **Create**
- **Delete**
- **Truncate**
- **Write** — at write pointer location
- **Read** — at read pointer location
- **Seek** — reposition within file
- **Open (F_i)** — search the directory structure on disk for entry F_i , and move the content of entry to memory
- **Close (F_i)** — move the content of entry F_i in memory to directory structure on disk

- Sequential Access

- Modeled on “tape” access modes. Works well on both sequential and random access devices
- Operations: `read_next()`; `write_next()`; `rewind()`. Possibly: `skip_forward()`; `skip_backward()`.
- Read and write operations occur at the current position; position is advanced automatically.



- Direct Access

- File modeled as a numbered sequence of blocks (or records). Operations now might depend on block number n
- Operations: `read(n)`; `write(n)`; `position_to(n)`; `read_next()`; `write_next()`; `rewrite(n)`

sequential access	implementation for direct access
<i>reset</i>	<code>cp = 0;</code>
<i>read next</i>	<code>read cp;</code> <code>cp = cp + 1;</code>
<i>write next</i>	<code>write cp;</code> <code>cp = cp + 1;</code>

- File owner/creator should be able to control:
 - What can be done
 - By whom
- Types of access
 - Read
 - Write
 - Execute
 - Append
 - Delete
 - List
 - Attribute change

Access Lists and Groups in Unix

- Mode of access: read, write, execute
- Three classes of users on Unix / Linux

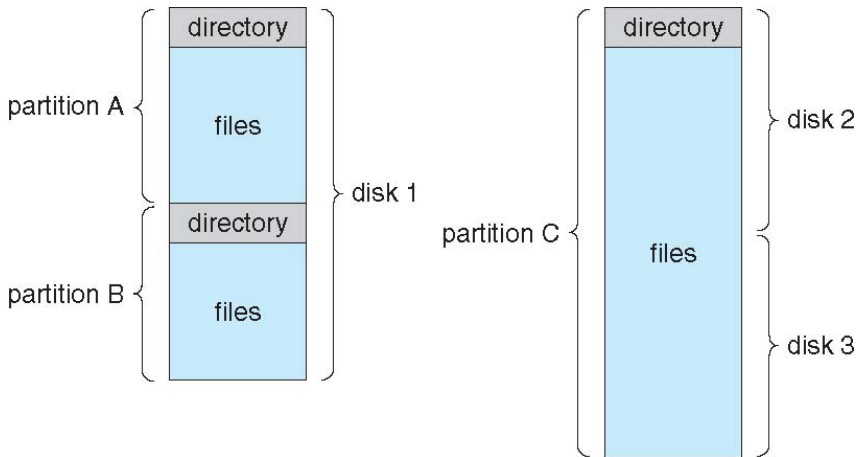
Class	Octal	r	w	x	RWX
user (owner)	7	1	1	1	RWX
group	6	1	1	0	RW-
others (public, anyone else)	1	0	0	1	--X

- Change file permission of a file: `chmod <permissions> <file>`

owner group public
 | | |
 └─┬───┘
 chmod 761 game

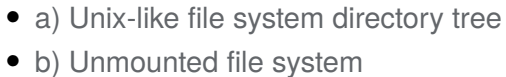
- Attach a group to a file: `chgrp <group> <file>`

A Typical File-system Organization

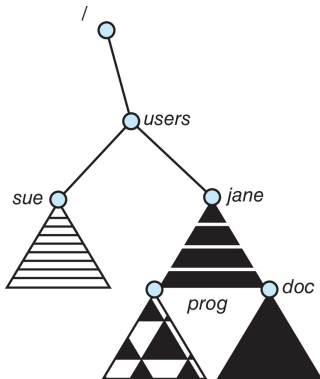


- tmpfs — memory-based volatile FS for fast, temporary I/O
- devpts — device management filesystem; automatically creates device nodes in /dev for hardware devices detected by the kernel
- sysfs — virtual filesystem providing information about various kernel subsystems, hardware devices, and associated device drivers
- securityfs — a virtual filesystem for security-related information, including SELinux
- pstore — persistent storage for kernel crash logs
- efivars — interface to the UEFI firmware variables
- bpf — Berkeley Packet Filter filesystem for storing BPF programs and maps
- configfs — a RAM-based filesystem that provides a means to configure kernel objects
- proc — process and system information as a filesystem
- selinuxfs — SELinux status and configuration
- debugfs — kernel debugging information
- mqueue — POSIX message queues
- tracefs — kernel tracing information
- hugetlbfs — lets applications use mmaped files explicitly backed by huge pages
- fusectl — control interface for FUSE (Filesystem in Userspace) filesystems
- squashfs — compressed read-only filesystem for embedded systems
- binfmt_misc — lets root associate handlers to arbitrary executable file formats
- rpc_pipefs — used by the kernel to export named pipes for remote procedure calls
- ext4, btrfs, zfs, ... — **general purpose file systems**
- ...

UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA



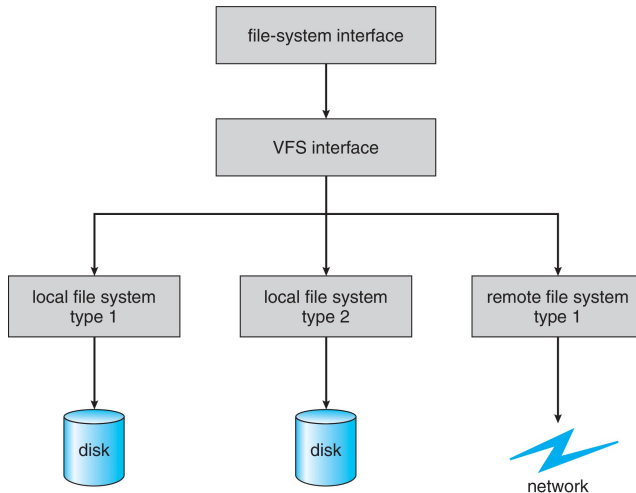
- After mounting (b) into the existing directory tree (/users is the mountpoint for filesystem b)



- Virtual File Systems (VFS) on Unix provide an object-oriented way of implementing file systems
- VFS allows the same system call interface (the API) to be used for different types of file systems
 - Separates file-system generic operations from implementation details
 - Implementation can be one of many file systems types, or network file system

Virtual File Systems (Cont.)

- The API is to the VFS interface, rather than any specific type of file system
- VFS layer translates the generic file operation into the specific file system operation

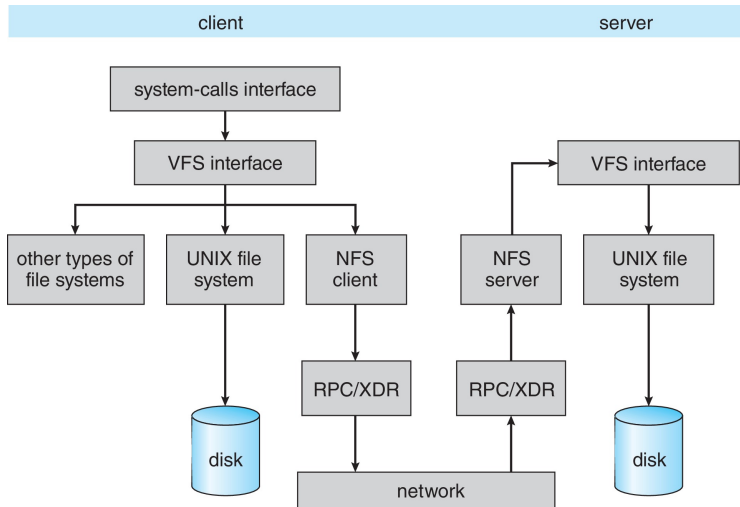


- For example, Linux has four object types: inode, file, superblock, dentry (directory entry)
- VFS defines set of operations on the objects that must be implemented
- Every object has a pointer to a function table
- Function table has addresses of routines to implement that function on that object
 - `int open(...)` → Open a file
 - `int close(...)` → Close an already-open file
 - `ssize_t read(...)` → Read from a file
 - `ssize_t write(...)` → Write to a file
 - `int mmap(...)` → Memory-map a file

- Sharing of files across a network
- First method involved manually sharing each file – programs like ftp
- Second method uses a distributed file system (DFS)
 - Remote directories visible from local machine
- Third method – World Wide Web
 - A bit of a revision to first method
 - Use browser to locate file/files and download /upload

- Sharing between a server (providing access to a file system via a network protocol) and a client (using the protocol to access the remote file system)
- Identifying each other via network ID can be spoofed, encryption can be performance expensive
- NFS an example
 - User auth info on clients and servers must match (UserIDs for example)
 - Remote file system mounted, file operations sent on behalf of user across network to server
 - Server checks permissions, file handle returned
 - Handle used for reads and writes until file closed

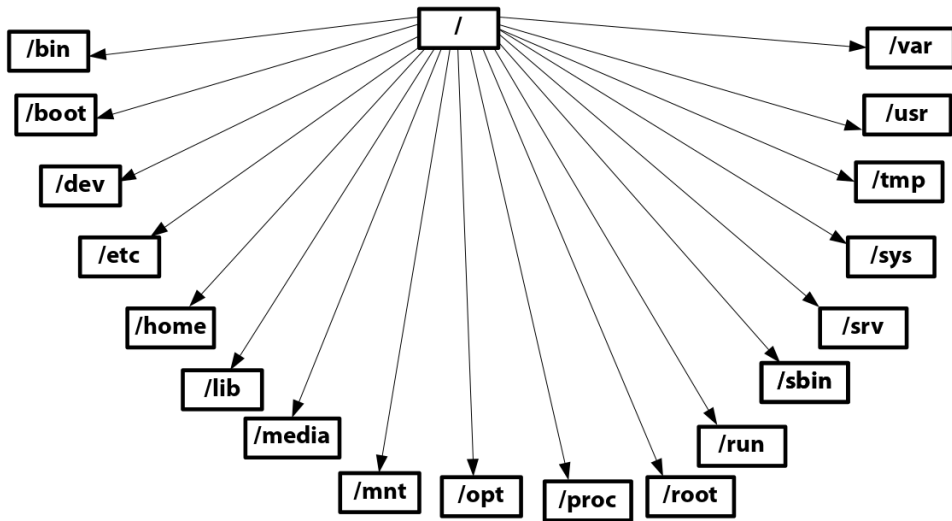
Schematic View of NFS Architecture



Filesystem hierarchy standard in Linux

- The Filesystem Hierarchy Standard (FHS) defines the directory structure and directory contents in Linux and other Unix-like operating systems
- FHS is maintained by the Linux Foundation
https://refspecs.linuxfoundation.org/FHS_3.0/fhs-3.0.html
- Cross-distribution compatibility → making it easier for users and developers to navigate and manage the file system across different Linux distributions

Filesystem hierarchy standard in Linux



- **/bin** — essential command binaries, everything needed for a base, text-based (no GUI) system
- **/boot** — static files of the boot loader, kernel images and their configurations
- **/dev** — device special files, such as **/dev/sda** for the first hard disk, **/dev/tty** for the terminal, **/dev/null** for the null device, ...
 - Device files are interfaces to device drivers that appear in a file system as if they were ordinary files. They allow software to interact with hardware devices by reading and writing to these special files.
- **/etc** — host-specific system configuration, such as network configuration, configuration of system services, ...
 - Contains configuration files that control the behavior of the system and installed applications. These files are typically text-based and can be edited by the system administrator to customize the system's operation.

- **/home** — user home directories, where users store their personal files and settings. Each user typically has a subdirectory under /home, named after their username (e.g., **/home/alice** for user "alice").
- **/lib** — essential shared libraries and kernel modules, needed to boot the system and run the commands. Shared libraries are collections of pre-compiled code that can be used by multiple programs, allowing for code reuse and efficient memory usage. Kernel modules are pieces of code that can be loaded into the kernel to extend its functionality, such as device drivers or file system support. **/lib64** is a common subdirectory for 64-bit libraries on 64-bit systems.
- **/media** — mount point for removable media, such as CD-ROMs, USB drives, flash memories, ...
 - When a removable media device is connected to the system, it can be automatically mounted to a subdirectory under /media (e.g., **/media/usb** for a USB drive). This allows users to access the contents of the removable media

- **/mnt** — mount point for mounting a filesystem temporarily, such as during system maintenance or for testing purposes. Unlike /media, which is typically used for removable media, /mnt is intended for temporary mounts of filesystems that are not automatically managed by the system.
- **/opt** — add-on application software packages, such as third-party applications that are not part of the base system. This directory is often used for software that is installed manually by the system administrator, rather than through the package manager.

- **/proc** — virtual filesystem providing process and kernel information as files, such as **/proc/cpuinfo** for CPU information, **/proc/meminfo** for memory information, **/proc/PID** for information about a specific process with Process ID *PID*, ...
 - The **/proc** filesystem is a special virtual filesystem that provides a mechanism for the kernel to expose information about processes and system resources in a hierarchical file-like structure. It allows users and applications to access real-time information about the system's state and processes by reading from files within the **/proc** directory.
- **/root** — home directory for the root user, the superuser with administrative privileges. This directory is separate from the **/home** directory to ensure that the root user's files and settings are kept distinct from those of regular users.
 - The root user has unrestricted access to all files and commands on the system, and the **/root** directory serves as the root user's personal workspace for storing files, configuration settings, and scripts that are specific to administrative tasks.

- **/run** — runtime variable data, such as information about currently logged-in users, running services, and system state. This directory is typically used for storing transient data that is generated during the system's operation and does not need to persist across reboots.
- **/sbin** — essential system binaries, such as system administration commands that are typically used by the root user for tasks like system maintenance, network configuration, and troubleshooting. These binaries are considered essential for the proper functioning of the system and are usually not intended for use by regular users.
 - Examples of binaries in **/sbin** include **/sbin/ifconfig** for network configuration, **/sbin/reboot** for rebooting the system, and **/sbin/fsck** for checking and repairing file systems.

- **/srv** — data for services provided by the system, such as web server data, FTP server data, and other service-specific data. This directory is intended for storing data that is served by the system's services and is typically organized into subdirectories based on the service name (e.g., **/srv/www** for web server data).
 - The /srv directory provides a standardized location for service-related data, making it easier for administrators to manage and locate files associated with specific services.

- **/sys** — virtual filesystem providing information about devices, drivers, and some kernel features, such as **/sys/class** for device classes, **/sys/block** for block devices, **/sys/bus** for device buses, ...
 - The /sys filesystem is a special virtual filesystem that provides a structured interface to the kernel's device model. It allows users and applications to access information about hardware devices, their drivers, and various kernel features by reading from files within the /sys directory.
 - differently from /dev, which provides an interface to device drivers for user-space applications to interact with hardware devices, /sys provides a more detailed and structured view of the system's hardware and kernel features, making it useful for system monitoring, debugging, and configuration tasks.

- **/tmp** — temporary files. The /tmp directory is typically cleared on system reboot, and it is intended for short-term storage of files that do not need to persist across reboots.
 - Applications can use the /tmp directory to store temporary data, such as session information, cache files, or intermediate results. Users can also create temporary files in this directory for various purposes, but they should be aware that these files may be deleted automatically when the system is restarted.
- **/usr** — secondary hierarchy for read-only user data. This directory is typically used for storing software that is installed on the system, including user applications, libraries, documentation, and other resources. The /usr directory is often organized into subdirectories such as **/usr/bin** for user binaries, **/usr/lib** for libraries, and **/usr/share** for shared data.
 - The /usr directory is intended for software that is not essential for the basic operation of the system but is commonly used by users and applications. It is often read-only for unprivileged users.

- **/var** — variable data files. This directory is intended for storing data that is expected to change during the operation of the system, such as log files, temporary files, and other data that is not essential for the system's basic operation.
 - Applications can use the /var directory to store variable data, such as log files, cache files, or other temporary data that needs to persist across reboots. Users can also create variable data in this directory for various purposes. However, they should be aware that this data may grow over time and may need to be managed to prevent it from consuming too much disk space.
 - Common subdirectories within /var include **/var/log** for log files, **/var/tmp** for temporary files that should persist across reboots, and **/var/spool** for spool files used by various services (e.g., mail spools, print spools).